

Práctica N° 1 – SQL Server – Sentencias DDL

Un negocio vende placas de madera enteras y por recortes

Se dispone de las siguientes tablas para almacenar la información de las ventas:

```
CREATE TABLE dbo.Cliente (id_cliente int NOT NULL IDENTITY (1,1) PRIMARY KEY, NombreYApellido varchar(50), DNI int, FechaNacimiento date)
```

```
CREATE TABLE dbo.Producto (ID_Producto int NOT NULL IDENTITY (1,1) PRIMARY KEY, nombre varchar(50) NULL, precio_plancha smallmoney NULL, precio_m2 smallmoney NULL, largo_cm smallint NULL, ancho_cm tinyint NULL, superficie_m2 decimal(5,2) NULL, stock_minimo smallint NOT NULL, foto varbinary(max))
```

```
CREATE TABLE dbo.His_Producto (ID_Producto int NOT NULL, precio_plancha smallmoney NULL, fecha_hasta date NOT NULL)
```

```
CREATE TABLE dbo.Factura ( numero_factura int NOT NULL PRIMARY KEY, fechahora smalldatetime NOT NULL, id_cliente int NOT NULL REFERENCES dbo.Cliente(id_cliente), montototal smallmoney NULL )
```

```
CREATE TABLE dbo.Renglon_factura (numero_factura int not null REFERENCES dbo.factura(numero_factura), numero_renglon tinyint, id_producto int REFERENCES dbo.Producto(ID_Producto) , cantidad smallint, largo_cm tinyint, ancho_cm tinyint, preciounitario smallmoney, preciototal money)
```

```
CREATE TABLE dbo.Compras_Producto (id_producto int not null REFERENCES dbo.Producto(ID_Producto), fecha smalldatetime not null, cantidad int not null)
```

```
CREATE TABLE dbo.Pedidos_Producto (id_producto int not null REFERENCES dbo.Producto(ID_Producto), fecha smalldatetime not null, cantidad int not null)
```

```
CREATE TABLE dbo.Recepcion_Producto (id_producto int not null REFERENCES dbo.Producto(ID_Producto), fecha smalldatetime not null, cantidad int not null, fecha_procesado smalldatetime null)
```

```
CREATE TABLE dbo.Proveedor (id_proveedor int NOT NULL IDENTITY (1,1) PRIMARY KEY, nombre varchar(50) NULL , fecha_lista_precios smalldatetime not null, productos_XML XML, productos_JSON VARCHAR(MAX))
```

1) Funciones

- Crear una función llamada SuperficieM2 que devuelva la superficie en m2 de un producto recibiendo 2 parámetros (largo y ancho)
- Crear una función llamada PrecioRecorte que calcule el precio de un recorte de una placa recibiendo como parámetros el Id_Producto, largo y el ancho, tomando como base el precio del metro cuadrado de placa.
Nota1: Tener en cuenta que el recorte debe ser menor que el tamaño de la placa.
Nota2: Contemplar si alguno de los parámetros que se reciben tiene valor nulo (null)
- Crear una función que devuelva el stock de un determinado producto.
Nota: se puede obtener de restar el total vendido que se obtiene de la tabla renglon_factura al total comprado que se obtiene de la tabla recepcion_producto
- Crear una función que devuelva para un producto determinado, el historial de pedidos, compras y recepciones ordenados por fecha, con los siguientes campos: nombre_producto, fecha, tipo_movimiento , cantidad. Nota: los posibles valores del campo tipo_movimiento serían 'Compra'/'Pedido'/'Recepcion'
- Crear una función que reciba una fecha y devuelva la cantidad de años que pasaron desde esa fecha hasta la actualidad.

2) Columnas calculadas y valores por defecto

- a) Expresar como sería la sentencia de creación de la tabla producto para que la columna superficie sea una columna calculada. Indique las dos alternativas posibles de definir la columna superficie como calculada
- b) Indicar la sentencia que debe ejecutarse para lograr que la columna fechahora de la tabla factura tenga por defecto el valor de la fecha y hora actual

3) Restricciones

Crear las siguientes restricciones para asegurar que:

- a) La cantidad vendida de un producto no sea ≤ 0
- b) El largo y ancho de un producto no sea ≤ 0
- c) El DNI del cliente no se repita
- d) La edad de un cliente sea mayor de 18 años.

4) Vistas

- a) Crear una vista que devuelva los siguientes datos: el número y la fecha de la factura, el nombre del cliente, la cantidad, el nombre y el precio total del producto.

5) Triggers

- a) Crear un trigger para que cuando se inserte y/o actualice un producto si la superficie es 0, al campo **precio_m2** se le fuerce el valor 0
- b) Agregar una columna “stock” de tipo integer en la tabla producto. Actualizar para todos los registros el campo creado con el stock que devuelve la función creada en el punto 1)c).
- c) Crear un trigger que cuando se venda un producto actualice el campo “stock” y si el stock es inferior al mínimo se inserte un registro en la tabla Pedidos_producto con una cantidad igual a la última compra registrada en Compras_producto
- d) Crear un trigger que cuando se modifique el precio de la plancha de un producto, guarde el precio anterior en la tabla his_producto.

6) Procedimientos almacenados

- a) Crear un SP que reciba todos los parámetros necesarios para efectuar la registración de una factura, efectúe las validaciones necesarias y devuelva el número de factura recientemente creado.
- b) Crear un SP que reciba todos los parámetros necesarios para efectuar la registración de un renglón de una factura y actualice el importe total de la factura.
- c) Crear un procedimiento almacenado que reciba todos los parámetros necesarios para efectuar la registración de una factura, efectúe las validaciones necesarias y devuelva el número de factura recién generado. NOTA: debe recibirse en un único parámetro el conjunto de renglones de la factura.
- d) Crear un SP que reciba todos los parámetros necesarios para insertar un producto, incluyendo la foto del producto.

NOTA: para probarlo y cargar un archivo jpg en una variable deben usar la sentencia OPENROWSET

7) Cursores

Crear un SP que actualice la información de los pedidos pendientes y los productos comprados tomando como base la información en la tabla **Recepción_Producto** que tiene los productos que se han recibido. Lo Se debe generar un **cursor** con todos los registros existentes en la tabla **Recepción_Producto**, y en la iteración sobre cada uno de los registros hacer lo siguiente: insertar un registro en la tabla **Compras_Producto** con la misma fecha y cantidad que en la tabla **Recepcion_Producto**, y si el producto existe en la tabla **Pedidos_Producto** con la misma cantidad, eliminar el registro en **Pedidos_Productos**, y en caso que la cantidad recepcionada sea menor que la de la tabla Pedidos, le decremente dicha cantidad a la tabla **Pedidos**.

Además, se debe incrementar el stock del producto en la cantidad indicada por el campo **cantidad** del registro actual en tratamiento. Por ultimo setear el campo **fecha_procesado** con la fecha/hora actual.

El efecto de este SP es que:

- Si había un pedido pendiente para un producto y se recibió completo, se lo dé por satisfecho (se elimina el registro) y se registre como producto comprado (al insertar en la tabla **Compras_Producto**)
- Si había un pedido pendiente para un producto y se recibió menos cantidad, siga figurando un pedido con la cantidad disminuida según lo recibido y se registre como producto comprado (al insertar en la tabla **Compras_Producto**)
- Se actualice el stock de los productos en función de las cantidades recibidas
- Se marque como “procesado” c/u de los registros de la tabla **Recepcion_Producto** para no volverlos a procesar

8) **Transacciones:** Modifique el SP del ejercicio 6)b) para que la operación se realice como una transacción.

a) Utilizando la variable @@error

b) Utilizando el manejo de errores con TRY..CATCH

Nota: se deben realizar dos versiones de procedimientos diferentes

9) Transacciones – concurrencia y aislamiento

Tomando como base el ejercicio 6)c), y suponiendo el escenario frecuente de que hubiera dos o mas conexiones a la bd que ejecutaran simultáneamente el procedimiento que crea una factura, realizar las modificaciones necesarias en el SP para garantizar que no pueda repetirse el número de factura.

Nota: usar modos de aislamiento y sugerencias de bloqueo

10) XML

a) Escribir una consulta que genere un XML con información de todos los productos e incluya los siguientes atributos: ID_Producto, nombre, Precio_plancha, largo_cm, ancho_cm

Genere dos variantes distintas de XML: uno con elementos y atributos; y otra solo elementos (sin atributos)

Insertar la variante sin atributos en la tabla **Proveedor** en el campo **productos_XML**.

b) Crear una copia de la tabla producto (producto_copiaXML) y escribir una consulta que inserte en dicha tabla los registros del archivo XML insertados en el punto a)

11) JSON

a) Generar un JSON con la misma información que el punto 10)a) e insertarlo en la tabla **Proveedor** (para otro Id_Proveedor) en el campo **productos_JSON**.

b) Crear una copia de la tabla producto (producto_copiaJSON) y escribir una consulta que inserte en dicha tabla los registros del archivo JSON insertados en el punto a)

12) JSON y XML

a) Escribir una consulta que aumente el precio de un producto existente en la lista de productos XML de un proveedor

b) Escribir una consulta que aumente el precio de un producto existente en la lista de productos JSON de un proveedor